

Compte Rendu Technique

SAÉ G3



Équipe Projet	Encadrants
Rayane Cathelin	Francis Garcia
Lilian Martin	Fabien Laguillaumie

Université de Montpellier – IUT de Montpellier – Département Informatique
Année universitaire : 2025-2026

SOMMAIRE

1. Contexte et Objectifs de la SAE.....	2
2. Choix Technologiques et Justifications.....	4
2.1 Conteneurisation avec Docker.....	4
2.2 Orchestration avec Docker Compose.....	4
3. Architecture Générale de l'Infrastructure.....	6
3.1 Vue d'Ensemble.....	6
3.2 Architecture Réseau.....	6
3.3 Persistance des Données.....	7
4. Gestion des Identités et Authentification Centralisée.....	8
4.1 OpenLDAP : Annuaire Central.....	8
4.2 Authentik : Fournisseur d'Identité (IAM).....	8
4.3 Automatisation.....	9
5. Services Applicatifs Déployés.....	10
5.1 Nextcloud – Stockage et Partage de Fichiers.....	10
5.2 Mattermost – Messagerie Collaborative.....	10
5.3 Portainer – Administration de l'Infrastructure.....	10
6. Accès aux Services et Sécurisation.....	12
6.1 Limites de l'Approche Locale (localhost).....	12
6.2 Mise en Place d'un Nom de Domaine.....	12
6.3 Sécurisation avec Cloudflare Tunnel.....	13
7. Sécurité Globale de l'Infrastructure.....	14
7.1 Centralisation de l'authentification et réduction de l'exposition des identifiants.....	14
7.2 Isolation réseau et protection des composants internes.....	14
7.3 Reverse proxy, HTTPS et cohérence des accès utilisateurs.....	14
7.4 Transmission du contexte de connexion aux applications.....	15
7.5 Cas de Nextcloud et intégration OpenID Connect.....	16
7.6 Cas de Mattermost et gestion de la confiance proxy.....	16
7.7 Sécurisation renforcée de Portainer et exposition maîtrisée.....	17
8. Bilan et Apports de la SAE.....	18
9. Perspectives d'Amélioration.....	19
Annexes :.....	20
Annexe 1 : docker-compose.yml.....	20
Annexe 2 : Mattermost Provider.....	27
Annexe 3 : Nextcloud Provider.....	28
Annexe 4 : Portainer Provider.....	28
Annexe 5 : Config Mattermost.....	29
Annexe 6 : Config Nextcloud.....	29
Annexe 7 : Config Portainer.....	30

1. Contexte et Objectifs de la SAE

Cette SAE s'inscrit dans le cadre du semestre de troisième année du BUT Informatique et a pour objectif de mobiliser et de consolider l'ensemble des compétences techniques acquises tout au long de la formation. Elle met particulièrement l'accent sur l'administration des systèmes, la virtualisation et la conteneurisation des services, la gestion des réseaux ainsi que les problématiques de sécurité des systèmes d'information. Le projet permet également d'aborder de manière concrète le déploiement et l'exploitation de services applicatifs dans un environnement maîtrisé.

L'objectif principal de cette SAE est la conception et la mise en œuvre d'une infrastructure complète de services collaboratifs, comparable à celle que l'on peut rencontrer dans un contexte professionnel ou en entreprise. Il ne s'agit pas uniquement de faire fonctionner des services de manière isolée, mais de les intégrer au sein d'une architecture cohérente et fonctionnelle, prenant en compte des contraintes réalistes d'accès, de sécurité et de maintenance.

Dans cette optique, le projet vise à déployer plusieurs services applicatifs hétérogènes, reposant sur des technologies différentes, tout en garantissant leur interconnexion au sein d'une même infrastructure. Un enjeu majeur du projet réside dans la centralisation de l'authentification des utilisateurs, afin d'offrir une expérience unifiée et sécurisée grâce à un mécanisme de Single Sign-On. Cette approche permet de réduire la multiplication des identifiants, d'améliorer la gestion des accès et de renforcer la sécurité globale du système.

La sécurisation des accès constitue également un axe central de cette SAE. L'infrastructure doit limiter l'exposition des services sensibles, protéger les données échangées et garantir que seuls les utilisateurs autorisés puissent accéder aux différentes applications. Par ailleurs, le projet intègre la problématique de l'accès aux services depuis plusieurs machines du réseau, voire depuis l'extérieur, ce qui implique la mise en place de mécanismes d'exposition sécurisée et de résolution des limitations liées à un accès local de type *localhost*.

Enfin, une attention particulière est portée à la persistance et à la maintenabilité des données et des services. L'infrastructure doit être capable de résister aux redémarrages, de faciliter les mises à jour et de permettre une maintenance efficace. L'ensemble de ces objectifs vise à proposer une solution fonctionnelle, réaliste et conforme à plusieurs bonnes pratiques actuelles en matière d'administration et de déploiement d'infrastructures informatiques.

La mise en place de cette infrastructure s'est faite de manière progressive. Le projet a nécessité plusieurs phases de tests, d'erreurs et de corrections, notamment lors de l'intégration des services, de la configuration du reverse proxy, de la mise en place du nom de domaine et du tunnel Cloudflare, ainsi que de l'implémentation de l'authentification centralisée. Ces ajustements ont permis d'aboutir à une solution fonctionnelle et cohérente.

Plusieurs choix techniques ont été ajustés en cours de projet afin de résoudre des problèmes de compatibilité, de sécurité ou de fonctionnement, ce qui reflète une démarche réaliste d'administration système.

2. Choix Technologiques et Justifications

2.1 Conteneurisation avec Docker

Le choix de Docker comme technologie centrale de déploiement repose sur sa capacité à fournir un environnement d'exécution isolé, reproductible et indépendant du système hôte. La conteneurisation permet d'encapsuler chaque service applicatif avec l'ensemble de ses dépendances, garantissant ainsi un comportement identique quel que soit l'environnement de déploiement. Cette caractéristique est particulièrement importante dans le cadre d'une SAE, où la reproductibilité et la maîtrise de l'environnement technique sont essentielles.

L'utilisation de Docker permet également une isolation claire entre les différents services de l'infrastructure. Chaque application s'exécute dans son propre conteneur, ce qui limite les interférences et les conflits de dépendances. Cette séparation facilite la maintenance de l'infrastructure, car un dysfonctionnement ou une mise à jour d'un service n'impacte pas directement les autres composants du système. De plus, la gestion du cycle de vie des services est simplifiée : un conteneur peut être arrêté, redémarré ou recréé de manière indépendante.

Un autre avantage majeur de Docker réside dans la portabilité de l'infrastructure. L'ensemble des services peut être déployé sur n'importe quelle machine disposant de Docker, sans nécessiter de configuration complexe du système d'exploitation. Cette portabilité rapproche l'infrastructure des standards professionnels actuels, largement basés sur la conteneurisation pour le déploiement d'applications.

Enfin, Docker contribue à la standardisation de l'environnement d'exécution. Tous les services partagent un socle technique cohérent, ce qui réduit les écarts de configuration et facilite la compréhension globale de l'architecture. Cette approche favorise également l'automatisation et s'inscrit dans une démarche DevOps moderne.

2.2 Orchestration avec Docker Compose

L'orchestration de l'infrastructure est assurée par Docker Compose, qui permet de décrire l'ensemble des services et de leurs interactions au sein d'un fichier unique. Ce choix s'explique par la taille et la complexité maîtrisée du projet, pour lesquelles Docker Compose constitue une solution à la fois efficace et adaptée. Contrairement à des solutions plus complexes comme Kubernetes, Docker Compose offre une prise en main plus simple tout en répondant pleinement aux besoins fonctionnels de la SAE.

Le fichier `docker-compose.yml` joue un rôle central dans l'architecture. Il permet de définir de manière déclarative les services déployés, les réseaux utilisés, les volumes de persistance ainsi que les dépendances entre les conteneurs. Cette description globale améliore la lisibilité de l'infrastructure et facilite sa compréhension, tant pour le développeur que pour un tiers amené à reprendre le projet.

L'utilisation de Docker Compose permet également d'automatiser le déploiement de l'ensemble de l'infrastructure à l'aide de commandes simples. Le lancement ou l'arrêt des services se fait de manière centralisée, en s'appuyant sur les dépendances déclarées entre

les conteneurs. Cette automatisation réduit les risques d'erreurs liées à des manipulations manuelles et améliore la reproductibilité du système.

Bien que l'architecture générale repose sur une approche modulaire à base de conteneurs Docker, son implémentation a nécessité plusieurs ajustements au cours du projet. L'intégration progressive des services, notamment du reverse proxy Traefik et du fournisseur d'identité Authentik, a impliqué des phases de test et de reconfiguration afin d'assurer une communication cohérente entre les conteneurs et une exposition correcte des services vers l'extérieur.

Enfin, Docker Compose s'intègre naturellement dans une démarche d'Infrastructure as Code. L'architecture complète de l'infrastructure est décrite sous forme de fichiers de configuration versionnables, ce qui facilite les évolutions, le travail collaboratif et la traçabilité des modifications.

Cette approche reste cependant limitée par l'utilisation de Docker Compose, qui ne permet pas une orchestration avancée (scalabilité automatique, haute disponibilité ou gestion dynamique des services), contrairement à des solutions comme Kubernetes.

3. Architecture Générale de l'Infrastructure

3.1 Vue d'Ensemble

L'infrastructure mise en place repose sur une architecture **multi-services**, organisée autour d'un fournisseur d'identité centralisé. Cette organisation permet de découpler les différentes briques fonctionnelles tout en assurant leur intégration au sein d'un système cohérent. Chaque service remplit un rôle précis et interagit avec les autres selon des flux définis progressivement au cours du projet, à mesure des phases de test et d'ajustement.

Au cœur de cette infrastructure se trouve Authentik, qui assure la gestion des identités et de l'authentification des utilisateurs. Il joue un rôle transversal en assurant l'authentification centralisée des utilisateurs pour l'accès aux différents services applicatifs. OpenLDAP complète ce dispositif en fournissant un annuaire centralisé, simulant un environnement d'entreprise réel dans lequel les identités sont stockées et gérées de manière standardisée.

Autour de ces composants centraux gravitent plusieurs services applicatifs répondant à des besoins fonctionnels distincts. Nextcloud est utilisé comme solution de stockage et de partage collaboratif, Mattermost comme outil de communication interne, et Portainer comme interface d'administration de l'infrastructure Docker. Chaque service applicatif s'appuie sur ses propres mécanismes de persistance, généralement via une base de données dédiée.

Cette approche modulaire permet de faire évoluer l'infrastructure de manière progressive, dans les limites imposées par l'utilisation de Docker Compose. Un service peut être modifié, remplacé ou supprimé sans remettre en cause l'ensemble du système, sous réserve d'adapter les configurations communes telles que l'authentification ou le reverse proxy.

3.2 Architecture Réseau

L'ensemble des conteneurs est connecté à un réseau Docker privé de type *bridge*, nommé `infra_net`. Ce réseau constitue une couche d'abstraction essentielle, permettant de maîtriser précisément les communications entre les services. Les conteneurs peuvent communiquer entre eux via leurs noms de services, sans exposer directement leurs ports à l'extérieur de l'hôte, l'accès étant assuré via un reverse proxy.

Cette organisation assure une isolation efficace entre l'hôte et les conteneurs, limitant ainsi les risques liés à une exposition directe des services internes. Les communications inter-services s'effectuent exclusivement au sein du réseau Docker, ce qui renforce la sécurité et réduit la surface d'attaque globale de l'infrastructure. Seuls les services nécessitant un accès utilisateur sont exposés via des points d'entrée contrôlés, généralement à travers un reverse proxy.

L'exposition des services vers l'extérieur repose sur Traefik, utilisé comme reverse proxy central. Il assure le routage des requêtes en fonction des noms de domaine, la transmission du contexte de connexion aux applications et l'intégration avec les mécanismes de sécurisation mis en place.

Les bases de données constituent des composants particulièrement sensibles. Elles ne sont jamais exposées publiquement et ne sont accessibles que depuis les conteneurs applicatifs qui en dépendent. Ce choix architectural limite considérablement les risques d'intrusion ou de fuite de données et s'inscrit dans une démarche de sécurité par défaut, où l'accès est restreint au strict nécessaire.

3.3 Persistance des Données

La persistance des données est assurée par l'utilisation de volumes Docker, principalement sous forme de volumes nommés, et dans certains cas de bind-mounts. Ce mécanisme permet de conserver les données applicatives, les fichiers de configuration et les journaux lors du redémarrage ou de la recreation des conteneurs.

Les volumes nommés sont utilisés pour garantir une gestion fiable et isolée des données propres à chaque service, tandis que les bind-mounts sont employés lorsque l'accès direct aux fichiers depuis l'hôte est nécessaire, notamment pour l'inspection des configurations ou des journaux. Cette approche s'est révélée particulièrement utile lors des phases de test, de débogage et de correction des erreurs rencontrées au cours du projet.

Chaque service dispose de ses propres espaces de stockage, assurant une séparation claire des responsabilités et limitant les dépendances entre composants. Cette organisation contribue à la maintenabilité globale de l'infrastructure, en rendant chaque service identifiable et autonome sur le plan de la gestion des données.

4. Gestion des Identités et Authentification Centralisée

4.1 OpenLDAP : Annuaire Central

Un serveur OpenLDAP est mis en place afin de simuler un annuaire d'entreprise, élément central dans de nombreuses infrastructures professionnelles. L'utilisation d'un annuaire LDAP permet de centraliser les identités des utilisateurs dans un référentiel unique, indépendant des applications qui consomment ces informations. Cette séparation entre la gestion des identités et les services applicatifs constitue une bonne pratique largement répandue dans les systèmes d'information d'entreprise.

OpenLDAP fournit une base d'utilisateurs structurée et standardisée, reposant sur des schémas largement reconnus. Cette standardisation facilite l'interopérabilité avec d'autres systèmes et applications, qu'ils soient présents ou ajoutés ultérieurement à l'infrastructure. Le choix d'un annuaire LDAP permet également d'envisager des évolutions futures, comme l'intégration d'outils tiers ou la synchronisation avec d'autres sources d'identité.

Dans le cadre de cette SAE, OpenLDAP joue le rôle de source de référence pour les identités utilisateurs, utilisées par Authentik pour l'authentification et l'accès aux services. Il permet de dissocier la gestion des comptes utilisateurs de la logique applicative, ce qui renforce la cohérence et la maintenabilité de l'infrastructure. Ce choix reflète une architecture réaliste, proche de celles rencontrées dans un contexte professionnel, où les annuaires LDAP sont couramment utilisés pour centraliser les identités.

4.2 Authentik : Fournisseur d'Identité (IAM)

Authentik constitue le cœur de la gestion des accès et de la sécurité de l'infrastructure. Il agit comme un fournisseur d'identité (Identity and Access Management), en assurant l'authentification centralisée des utilisateurs et en fournissant les mécanismes nécessaires au Single Sign-On. L'ensemble des services applicatifs délègue ainsi l'authentification à Authentik, ce qui permet d'unifier les méthodes d'accès et de renforcer la sécurité globale.

Authentik assure la gestion des utilisateurs et des règles d'accès aux différentes applications, en s'appuyant sur des protocoles standards tels qu'OAuth2 et OpenID Connect. Cette approche garantit une compatibilité avec un large éventail d'applications et permet d'intégrer des services hétérogènes sans modification majeure de leur fonctionnement interne. Du point de vue utilisateur, cela se traduit par une expérience simplifiée, où une seule authentification suffit pour accéder à l'ensemble des services autorisés.

Sur le plan technique, Authentik repose sur une base de données PostgreSQL pour la persistance des informations et sur Redis pour la gestion du cache et l'amélioration des performances. Cette architecture permet de supporter efficacement les flux d'authentification tout en assurant la fiabilité et la cohérence des données. Le découplage entre les différents composants d'Authentik contribue également à la scalabilité et à la robustesse du système.

4.3 Automatisation

La configuration d'Authentik a été réalisée principalement via son interface d'administration web. Cette approche a permis de comprendre en profondeur le fonctionnement des mécanismes d'authentification, des flux OpenID Connect et des relations entre applications et fournisseurs d'identité.

Dans le cadre de cette SAE, une tentative d'utilisation des Blueprints, des fichiers de configuration déclaratifs rédigés au format YAML, a été réalisée. Toutefois, la complexité de mise en œuvre, combinée aux contraintes de temps et aux difficultés rencontrées lors de l'intégration avec les services existants, n'a pas permis d'aboutir à une configuration pleinement fonctionnelle par ce biais.

La configuration finale d'Authentik a donc été réalisée principalement via l'interface d'administration, tout en conservant une approche méthodique et reproductible. L'utilisation des Blueprints reste néanmoins identifiée comme une piste d'amélioration pertinente pour une industrialisation future de l'infrastructure.

5. Services Applicatifs Déployés

L'infrastructure mise en place repose sur plusieurs services applicatifs complémentaires, chacun répondant à un besoin précis d'un environnement d'entreprise. Ces services sont intégrés au sein d'une architecture conteneurisée commune et partagent un socle technique cohérent, notamment en matière de réseau, de persistance des données et d'authentification centralisée.

5.1 Nextcloud – Stockage et Partage de Fichiers

Nextcloud constitue le service de stockage collaboratif de l'infrastructure. Il permet aux utilisateurs de déposer, consulter et partager des fichiers au sein d'un espace centralisé, accessible depuis un navigateur web. Ce type de service répond à un besoin fondamental en entreprise, en facilitant la collaboration et l'échange de documents tout en conservant la maîtrise des données.

Sur le plan technique, Nextcloud repose sur une base de données dédiée et sur des volumes persistants pour le stockage des fichiers utilisateurs. Cette séparation entre les données applicatives et les conteneurs garantit la continuité du service en cas de redémarrage ou de mise à jour, et faciliterait la mise en place de stratégies de sauvegarde.

L'intégration de Nextcloud à l'architecture globale permet de démontrer la capacité de l'infrastructure à héberger des services manipulant des volumes de données importants, tout en restant compatibles avec une authentification centralisée et des contraintes de sécurité.

5.2 Mattermost – Messagerie Collaborative

Mattermost est déployé comme solution de communication interne entre les utilisateurs. Il offre des fonctionnalités de messagerie instantanée structurée, adaptées aux échanges professionnels et au travail en équipe. Ce service complète le stockage collaboratif en apportant un outil de communication temps réel au sein de l'infrastructure.

D'un point de vue technique, Mattermost s'appuie sur une base de données PostgreSQL dédiée et fonctionne comme un service applicatif indépendant au sein du réseau Docker privé. Son intégration à l'infrastructure a mis en évidence des problématiques spécifiques liées à l'exposition via un reverse proxy et à la gestion correcte des flux HTTPS, notamment lors de l'authentification.

Mattermost illustre ainsi les contraintes rencontrées lorsqu'un service web est placé derrière un proxy et intégré à un système d'authentification centralisée, et a permis de travailler sur des problématiques réalistes de confiance proxy et de cohérence des redirections.

5.3 Portainer – Administration de l'Infrastructure

Portainer est utilisé comme outil d'administration et de supervision de l'infrastructure Docker. Il fournit une interface graphique permettant de gérer les conteneurs, les images, les volumes et les réseaux, facilitant ainsi l'exploitation et le suivi de l'ensemble des services déployés.

Contrairement aux services applicatifs destinés aux utilisateurs finaux, Portainer est un outil sensible offrant des privilèges élevés sur l'infrastructure. Son exposition a donc fait l'objet d'une attention particulière en matière de sécurité. L'accès est strictement restreint aux utilisateurs autorisés, et le service est isolé au sein du réseau Docker, sans exposition directe de composants critiques.

Le déploiement de Portainer permet de démontrer la capacité à intégrer des outils d'administration dans une architecture sécurisée, tout en conservant une cohérence globale en matière d'accès, de réseau et de protection des services.

6. Accès aux Services et Sécurisation

6.1 Limites de l'Approche Locale (*localhost*)

Dans une première phase du projet, les services étaient accessibles uniquement via des adresses de type *localhost*. Cette approche a été utilisée lors des premières étapes de mise en place de l'infrastructure, notamment lors des tests effectués sur les machines de la salle informatique. Elle permettait un accès rapide aux services depuis la machine hôte, mais s'est rapidement révélée insuffisante dès lors qu'un accès depuis les autres postes de la salle a été envisagé.

L'utilisation de *localhost* limite en effet l'accès aux services à une seule machine, chaque poste interprétant cette adresse comme sa propre interface locale. Une première solution envisagée a consisté à ouvrir les ports des différentes applications afin de les rendre accessibles depuis le réseau local. Toutefois, cette approche soulevait plusieurs problèmes, notamment en termes de cohérence des URLs utilisées par les services.

Ces incohérences posaient des difficultés particulières pour la mise en place du Single Sign-On. Les mécanismes d'authentification OAuth2 et OpenID Connect reposent sur des redirections et des URLs strictement définies. Multiplier les ports et les points d'accès risquait de rendre ces flux d'authentification instables, voire incompatibles avec une authentification centralisée fiable.

Ces contraintes ont conduit à une remise en question de l'architecture d'accès initiale, et à la recherche d'une solution permettant de proposer des URLs uniques, stables et accessibles depuis l'ensemble des postes, tout en restant compatible avec les exigences de sécurité et d'authentification du projet.

6.2 Mise en Place d'un Nom de Domaine

Afin de lever les limitations liées à l'utilisation de *localhost*, l'infrastructure a été repensée autour de l'utilisation de noms de domaine dédiés pour chaque service. Cette approche permet de fournir des URLs uniques, stables et cohérentes, accessibles depuis n'importe quelle machine autorisée.

L'introduction de noms de domaine constitue une étape essentielle pour la mise en place d'un système d'authentification centralisée. Le Single Sign-On repose en effet sur des redirections et des callbacks nécessitant des URLs bien définies. En utilisant des domaines dédiés, l'infrastructure garantit une intégration correcte des mécanismes OAuth2 et OpenID Connect, tout en améliorant l'expérience utilisateur.

De plus, cette architecture basée sur des noms de domaine se rapproche fortement des pratiques observées en entreprise. Elle permet de séparer clairement les rôles des différents services et de faciliter leur identification par les utilisateurs, tout en offrant une meilleure évolutivité de l'infrastructure.

6.3 Sécurisation avec Cloudflare Tunnel

Pour renforcer la sécurité de l'infrastructure et permettre un accès distant maîtrisé, un Cloudflare Tunnel a été mis en place. Cette solution permet d'exposer les services hébergés localement sans ouvrir de ports entrants sur le réseau ou sur la box Internet. Le tunnel établit une connexion sortante sécurisée entre l'infrastructure et les serveurs Cloudflare, qui se chargent ensuite de relayer les requêtes des utilisateurs.

L'utilisation de Cloudflare Tunnel apporte plusieurs avantages majeurs en termes de sécurité. Le chiffrement HTTPS est géré automatiquement, sans configuration manuelle de certificats, ce qui garantit la confidentialité des échanges. De plus, l'infrastructure bénéficie des mécanismes de protection de Cloudflare, tels que la limitation des attaques par déni de service ou le filtrage de requêtes malveillantes.

La mise en place du tunnel a nécessité plusieurs ajustements, notamment lors de l'association avec les noms de domaine et les services exposés, avant d'aboutir à une configuration stable et fonctionnelle.

Grâce à cette solution, les services deviennent accessibles de manière sécurisée depuis l'ensemble du réseau, voire depuis Internet, tout en conservant un niveau de sécurité élevé. L'accès des utilisateurs s'effectue exclusivement via les domaines définis, en passant par Cloudflare, ce qui renforce la cohérence de l'architecture et réduit considérablement la surface d'attaque de l'infrastructure.

7. Sécurité Globale de l'Infrastructure

La sécurité de l'infrastructure constitue un axe central du projet réalisé dans le cadre de cette SAE. Elle n'a pas été envisagée comme une simple couche ajoutée en fin de conception, mais comme un ensemble de mécanismes intégrés progressivement au fil des phases de déploiement, de test et de correction. Les choix techniques opérés sont le résultat d'une démarche itérative, marquée par l'identification de dysfonctionnements concrets et par la mise en place de solutions adaptées aux contraintes réelles rencontrées.

7.1 Centralisation de l'authentification et réduction de l'exposition des identifiants

Le premier pilier de la sécurité repose sur la centralisation de l'authentification autour d'un fournisseur d'identité unique. Authentik est utilisé pour assurer l'authentification des utilisateurs pour l'ensemble des services applicatifs, tandis qu'OpenLDAP joue le rôle d'annuaire central contenant les identités. Cette séparation claire entre la gestion des identités, l'authentification et les services applicatifs permet de réduire considérablement les risques liés à la manipulation directe des mots de passe.

Dans cette architecture, les applications ne stockent ni ne vérifient les identifiants utilisateurs. Elles délèguent entièrement cette responsabilité à Authentik, qui délivre des jetons OAuth2 ou OpenID Connect après validation de l'identité. Ce mécanisme permet une gestion cohérente des accès, simplifie les révocations et renforce la sécurité globale du système.

7.2 Isolation réseau et protection des composants internes

La sécurité de l'infrastructure repose également sur une isolation réseau stricte. L'ensemble des conteneurs est interconnecté via un réseau Docker privé unique, ce qui permet aux services de communiquer entre eux sans exposer leurs ports internes à l'extérieur de l'hôte. Cette organisation empêche toute interaction directe non contrôlée entre les services internes et le réseau externe.

Les bases de données utilisées par les différents services applicatifs ne sont jamais exposées publiquement. Elles ne sont accessibles qu'aux conteneurs applicatifs qui en dépendent, via le réseau interne Docker. Cette approche limite fortement la surface d'attaque et protège les données sensibles contre des accès non autorisés depuis l'extérieur.

7.3 Reverse proxy, HTTPS et cohérence des accès utilisateurs

L'accès aux services applicatifs est assuré par un reverse proxy basé sur Traefik. Celui-ci permet de router les requêtes vers les bons services en fonction du nom de domaine utilisé, tout en centralisant la gestion des flux HTTP et HTTPS. Cette architecture facilite l'exposition contrôlée des services et permet d'imposer des règles de sécurité homogènes.

Cependant, le déploiement des applications derrière un reverse proxy a rapidement mis en évidence des problèmes de cohérence liés au protocole de connexion. Les applications communiquent avec Traefik en HTTP au sein du réseau Docker, ce qui les conduit par défaut à considérer que l'utilisateur accède également au service en HTTP. Cette situation a provoqué des erreurs de redirection, notamment lors des phases d'authentification OAuth et OpenID Connect, où des URLs incorrectes étaient générées.

Afin de garantir un accès sécurisé, une redirection systématique vers HTTPS a été mise en place pour les services exposés. L'extrait suivant illustre cette configuration pour le service Portainer :

- "traefik.http.middlewares.portainer-redirect.redirectscheme.scheme=https"
- "traefik.http.middlewares.portainer-redirect.redirectscheme.permanent=true"

Cette redirection garantit que les utilisateurs accèdent toujours aux services via une connexion chiffrée, condition indispensable à la sécurité des échanges et au bon fonctionnement des mécanismes d'authentification.

7.4 Transmission du contexte de connexion aux applications

Lors de l'intégration de l'authentification centralisée, plusieurs erreurs ont été observées après validation de l'identité côté fournisseur d'identité, notamment des refus d'accès ou des redirections incorrectes. L'analyse a montré que les applications ne disposaient pas des informations nécessaires pour déterminer le protocole, le domaine et le port réellement utilisés par les utilisateurs.

Pour résoudre ce problème, des en-têtes HTTP spécifiques ont été ajoutés via Traefik afin de transmettre explicitement le contexte réel de connexion aux applications. Ces en-têtes permettent aux services de générer des URLs cohérentes avec l'accès HTTPS et sont essentiels au bon fonctionnement des flux OAuth2 et OpenID Connect.

L'extrait suivant illustre cette configuration appliquée à Portainer, mais le même principe a été utilisé pour les autres services exposés :

- "traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Proto=https"
- "traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Host=portainer.rayanec.com"
- "traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Port=443"

Grâce à ces en-têtes, les applications interprètent correctement le contexte de connexion et évitent les incohérences lors des phases de redirection et de validation des jetons d'authentification.

7.5 Cas de Nextcloud et intégration OpenID Connect

Nextcloud a été intégré à l'infrastructure via le protocole OpenID Connect, en utilisant une extension dédiée. Cette application s'est révélée particulièrement sensible à la cohérence entre les URLs configurées côté fournisseur d'identité et celles générées par le service. Lors des premiers tests, plusieurs erreurs de redirection ont été rencontrées, liées à des divergences de protocole ou de nom de domaine.

La résolution de ces problèmes a nécessité une configuration rigoureuse des endpoints OpenID Connect ainsi qu'une prise en compte explicite du contexte HTTPS transmis par le reverse proxy. Une fois ces ajustements réalisés, l'authentification centralisée a pu fonctionner de manière stable, permettant aux utilisateurs d'accéder à Nextcloud sans gestion locale des identifiants ni duplication des comptes.

7.6 Cas de Mattermost et gestion de la confiance proxy

L'intégration de Mattermost a présenté des spécificités liées à son mécanisme d'authentification basé sur OpenID Connect. Mattermost dispose d'un support natif pour OpenID Connect, qui a été utilisé dans le cadre de cette SAE afin de déléguer l'authentification des utilisateurs à Authentik. Le service a été configuré à l'aide du point de découverte OpenID fourni par Authentik, ainsi que d'un couple `client_id` et `client_secret` dédié.

Lors de la mise en place, des dysfonctionnements ont été rencontrés lors de l'accès à Mattermost via le reverse proxy. Ces problèmes n'étaient pas liés au protocole OpenID Connect en lui-même, mais à la manière dont Mattermost interprétait les requêtes provenant d'un environnement proxifié en HTTPS. En l'absence de configuration explicite, le service ne faisait pas confiance aux en-têtes transmis par le proxy, ce qui entraînait une mauvaise détection du schéma de connexion et des erreurs lors des redirections d'authentification.

Afin de corriger ce comportement, Mattermost a été configuré pour faire confiance au reverse proxy en déclarant explicitement les adresses IP autorisées. Cette configuration permet au service de prendre en compte correctement les en-têtes transmis par le proxy, notamment ceux indiquant l'utilisation du protocole HTTPS. Une fois cette confiance établie, le processus d'authentification via OpenID Connect s'est déroulé de manière stable et cohérente, garantissant le bon fonctionnement du Single Sign-On pour les utilisateurs.

7.7 Sécurisation renforcée de Portainer et exposition maîtrisée

Portainer occupe une place particulière au sein de l'infrastructure en raison des privilèges élevés qu'il offre sur l'ensemble de l'environnement Docker. Toute mauvaise configuration ou exposition non maîtrisée représente un risque critique pour la sécurité globale.

Les difficultés rencontrées lors de son intégration ont mis en évidence la nécessité d'une configuration stricte combinant redirection HTTPS obligatoire, transmission correcte du contexte de connexion et configuration OAuth précise. Ces ajustements ont permis de garantir que l'interface d'administration n'est accessible qu'aux utilisateurs authentifiés et via des connexions chiffrées.

Enfin, l'exposition externe de l'ensemble des services est sécurisée par l'utilisation de Cloudflare Tunnel. Cette solution permet de rendre les services accessibles sans ouvrir de ports entrants sur le réseau local, en établissant une connexion sortante chiffrée vers l'infrastructure Cloudflare. Ce mécanisme limite fortement les risques liés à l'exposition directe des services et complète efficacement les mesures de sécurité mises en place au niveau du reverse proxy et de l'authentification centralisée.

8. Bilan et Apports de la SAE

Cette SAE a constitué une étape importante dans la consolidation des compétences techniques acquises au cours du BUT Informatique. Elle a permis de mobiliser de manière transversale des notions abordées dans différents enseignements, en les appliquant à un projet concret et cohérent. La conception et le déploiement d'une infrastructure complète ont offert une vision globale des enjeux liés à l'administration des systèmes et des réseaux, bien au-delà d'une simple mise en œuvre technique.

Le projet a notamment permis d'approfondir des compétences avancées en conteneurisation et en gestion de services conteneurisés. L'utilisation de Docker et Docker Compose a favorisé une meilleure compréhension des mécanismes d'isolation, de persistance des données et de gestion du cycle de vie des applications. Cette approche a également mis en évidence l'importance de la structuration de l'infrastructure et de la documentation, éléments essentiels dans un contexte professionnel.

La SAE a également mis en lumière les enjeux réels liés à la sécurité des systèmes d'information. La centralisation de l'authentification, la gestion des accès, l'isolation réseau et la sécurisation des flux ont montré que la sécurité ne peut pas être traitée comme un simple ajout, mais doit être intégrée dès la conception de l'architecture. La mise en place d'un fournisseur d'identité et l'utilisation de solutions comme Cloudflare Tunnel ont permis de comprendre concrètement les problématiques liées à l'exposition de services et à la protection des données.

Par ailleurs, la manipulation d'outils largement utilisés dans le monde professionnel, tels qu'Authentik, OpenLDAP, Nextcloud, Mattermost ou encore Portainer, a constitué un apport pédagogique majeur. Elle a permis de se familiariser avec des technologies réalistes et actuelles, tout en développant une méthodologie de travail rigoureuse, proche de celle attendue dans un environnement professionnel ou en entreprise.

Enfin, ce projet a permis de concevoir une infrastructure cohérente, évolutive et maintenable, intégrant des services hétérogènes autour d'une architecture commune. Il illustre une approche globale et réaliste de l'informatique, combinant conteneurisation, gestion des identités, sécurité réseau et accès distant sécurisé. Cette SAE représente ainsi une synthèse des compétences développées tout au long du semestre et constitue une expérience formatrice en vue d'une insertion professionnelle et d'une poursuite d'études dans le domaine de l'administration et de la sécurité des systèmes informatiques.

9. Perspectives d'Amélioration

Bien que l'infrastructure mise en place réponde aux objectifs initiaux de la SAE, plusieurs pistes d'amélioration peuvent être envisagées afin de la rendre encore plus robuste, sécurisée et proche des standards d'un environnement de production professionnel. Ces évolutions s'inscrivent dans une logique d'optimisation continue et de montée en maturité de l'infrastructure.

Une première amélioration concernerait la gestion fine des rôles et des permissions. Actuellement, l'authentification est centralisée et fonctionnelle, mais une politique plus détaillée de contrôle d'accès pourrait être mise en œuvre. L'exploitation avancée des groupes et des rôles dans Authentik permettrait de définir précisément les droits des utilisateurs selon leur profil, leur service ou leur niveau de responsabilité. Cette approche renforcerait la sécurité et améliorerait la gouvernance des accès, en accord avec les principes du moindre privilège.

Une autre perspective importante réside dans la mise en place d'une solution de supervision et de monitoring. L'intégration d'outils tels que Prometheus et Grafana permettrait de collecter et de visualiser des métriques sur l'état des services, l'utilisation des ressources et les performances globales de l'infrastructure. Cela offrirait une meilleure visibilité sur le fonctionnement du système, faciliterait la détection des incidents et contribuerait à une maintenance proactive, indispensable dans un contexte professionnel.

Une autre piste d'amélioration concerne l'automatisation de la configuration du fournisseur d'identité. Dans le cadre de cette SAE, la configuration d'Authentik a été réalisée principalement via l'interface d'administration web, ce qui a permis de comprendre en profondeur les mécanismes d'authentification et les flux OpenID Connect. Une tentative d'utilisation des Blueprints, mécanisme déclaratif proposé par Authentik, a également été engagée, sans toutefois aboutir à une configuration pleinement fonctionnelle dans le temps imparti.

La mise en œuvre complète des Blueprints représenterait une évolution pertinente vers une approche pleinement orientée Infrastructure as Code. Elle permettrait d'automatiser le déploiement et la configuration d'Authentik, d'améliorer la reproductibilité de l'infrastructure et de faciliter son industrialisation dans un contexte de production. Cette automatisation constituerait un gain significatif en termes de maintenance, de fiabilité et de traçabilité des configurations.

Enfin la question des sauvegardes constitue également un axe d'amélioration majeur. Bien que la persistance des données soit assurée par des volumes Docker, la mise en place de sauvegardes automatisées et planifiées permettrait de renforcer la résilience de l'infrastructure. Des mécanismes de sauvegarde régulière des bases de données et des données applicatives, accompagnés de tests de restauration, garantiraient une meilleure protection contre la perte de données ou les erreurs humaines.

Annexes :

Les annexes suivantes présentent des extraits de configuration et des captures d'écran illustrant les choix techniques décrits dans le rapport, notamment la configuration des fournisseurs d'identité, des services applicatifs et du déploiement Docker.

Annexe 1 : docker-compose.yml

services:

```
# =====
# AUTHENTIK
# =====
authentik-server:
  image: ghcr.io/goauthentik/server:2025.2.4
  container_name: authentik-server
  restart: unless-stopped
  command: server
  environment:
    AUTHENTIK_REDIS__HOST: authentik_redis
    AUTHENTIK_POSTGRES__HOST: authentik_db
    AUTHENTIK_POSTGRES__USER: authentik
    AUTHENTIK_POSTGRES__NAME: authentik
    AUTHENTIK_POSTGRES__PASSWORD: authentik
    AUTHENTIK_SECRET_KEY: "RVaD8YQuP7M3sK2eTZnYFy7zL4hrV8L7"
    AUTHENTIK_BOOTSTRAP_EMAIL: "admin@local"
    AUTHENTIK_BOOTSTRAP_PASSWORD: "admin1234"
    AUTHENTIK_BOOTSTRAP_USERNAME: "admin"
    AUTHENTIK_LISTEN__TRUSTED_PROXY_IP: "172.19.0.0/16"
    AUTHENTIK_WEB__WORKERS: "2"
  volumes:
    - authentik_media:/media
    - authentik_templates:/templates
  depends_on:
    - authentik_db
    - authentik_redis
```

```

networks:
  - infra_net
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.authentik.rule=Host(`auth.rayanec.com`)"
  - "traefik.http.routers.authentik.entrypoints=web"
  - "traefik.http.services.authentik.loadbalancer.server.port=9000"
  -
"traefik.http.middlewares.authentik-headers.headers.customrequestheaders.X-Forwarded-Proto=https"
-
"traefik.http.middlewares.authentik-headers.headers.customrequestheaders.X-Forwarded-Host=auth.rayanec.com"
  - "traefik.http.routers.authentik.middlewares=authentik-headers"

authentik-worker:
  image: ghcr.io/goauthentik/server:2025.2.4
  container_name: authentik-worker
  restart: unless-stopped
  command: worker
  environment:
    AUTHENTIK_REDIS__HOST: authentik_redis
    AUTHENTIK_POSTGRES__HOST: authentik_db
    AUTHENTIK_POSTGRES__USER: authentik
    AUTHENTIK_POSTGRES__NAME: authentik
    AUTHENTIK_POSTGRES__PASSWORD: authentik
    AUTHENTIK_SECRET_KEY: "RVaD8YQuP7M3sK2eTZnYFy7zL4hrV8L7"
  depends_on:
    - authentik_db
    - authentik_redis
  networks:
    - infra_net
  labels:
    - "traefik.enable=false"

authentik-proxy:
  image: ghcr.io/goauthentik/proxy:2025.2.4
  container_name: authentik-proxy
  restart: unless-stopped
  environment:
    AUTHENTIK_HOST: http://authentik-server:9000
    AUTHENTIK_INSECURE: "true"
    AUTHENTIK_TOKEN: "dev-outpost-token-123456789"
    AUTHENTIK_OUTPOST_ID: "local-proxy-1"
  depends_on:
    - authentik-server
  networks:
    - infra_net

```



labels:
- "traefik.enable=false"

authentik_db:
image: postgres:15
container_name: authentik_db
environment:
 POSTGRES_PASSWORD: authentik
 POSTGRES_USER: authentik
 POSTGRES_DB: authentik
volumes:
- authentik_db:/var/lib/postgresql/data
networks:
- infra_net

authentik_redis:
image: redis:7-alpine
container_name: authentik_redis
restart: unless-stopped
networks:
- infra_net

=====
LDAP
=====

ldap:
image: osixia/openldap:1.5.0
container_name: ldap
environment:
 LDAP_ORGANISATION: "SAE INFRA"
 LDAP_DOMAIN: "sae.test"
 LDAP_ADMIN_PASSWORD: "admin"
networks:
- infra_net
volumes:
- ./ldap/data:/var/lib/ldap
- ./ldap/config:/etc/ldap/slapd.d

phpldapadmin:
image: osixia/phpldapadmin:latest
container_name: phpldapadmin
environment:
 PHPLDAPADMIN_LDAP_HOSTS: "ldap"
 PHPLDAPADMIN_ALLOW: "true"
 PHPLDAPADMIN_HTTPS: "false"
networks:
- infra_net
labels:

```

- "traefik.enable=true"
- "traefik.http.routers.ldapadmin.rule=Host(`ldap.rayanec.com`)"
- "traefik.http.routers.ldapadmin.entrypoints=web"
- "traefik.http.services.ldapadmin.loadbalancer.server.port=80"
-
"traefik.http.middlewares.ldap-headers.headers.customrequestheaders.X-Forwarded-Proto=
https"
-
"traefik.http.middlewares.ldap-headers.headers.customrequestheaders.X-Forwarded-Host=
dap.rayanec.com"
- "traefik.http.routers.ldapadmin.middlewares=ldap-headers"

# =====
# NEXTCLOUD
# =====
nextcloud:
  image: nextcloud:28
  container_name: nextcloud
  restart: unless-stopped
  depends_on:
    - nc_db
  environment:
    MYSQL_HOST: nc_db
    MYSQL_DATABASE: nextcloud
    MYSQL_USER: nextcloud
    MYSQL_PASSWORD: nextcloud
    OVERWRITEHOST: cloud.rayanec.com
    OVERWRITEPROTOCOL: https
    TRUSTED_PROXIES: traefik
  volumes:
    - ./nextcloud/html:/var/www/html
    - ./nextcloud/data:/var/www/html/data
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.nextcloud.rule=Host(`cloud.rayanec.com`)"
    - "traefik.http.routers.nextcloud.entrypoints=web"
    - "traefik.http.services.nextcloud.loadbalancer.server.port=80"
    -
"traefik.http.middlewares.nextcloud-headers.headers.customrequestheaders.X-Forwarded-P
roto=https"
-
"traefik.http.middlewares.nextcloud-headers.headers.customrequestheaders.X-Forwarded-H
ost=cloud.rayanec.com"
-
"traefik.http.middlewares.nextcloud-headers.headers.customrequestheaders.X-Forwarded-P
ort=443"
- "traefik.http.routers.nextcloud.middlewares=nextcloud-headers"
networks:

```



```

- infra_net

nc_db:
  image: mariadb:11
  container_name: nc_db
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: rootpass
    MYSQL_DATABASE: nextcloud
    MYSQL_USER: nextcloud
    MYSQL_PASSWORD: nextcloud
  volumes:
    - ./nextcloud/db:/var/lib/mysql
  networks:
    - infra_net

# =====
# PORTAINER
# =====
portainer:
  image: portainer/portainer-ce:latest
  container_name: portainer
  restart: unless-stopped
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - portainer_data:/data
  networks:
    - infra_net
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.portainer.rule=Host(`portainer.rayanec.com`)"
    - "traefik.http.routers.portainer.entrypoints=web"
    - "traefik.http.services.portainer.loadbalancer.server.port=9000"
    - "traefik.http.middlewares.portainer-redirect.redirectscheme.scheme=https"
    - "traefik.http.middlewares.portainer-redirect.redirectscheme.permanent=true"

    - "traefik.http.routers.portainer-secure.rule=Host(`portainer.rayanec.com`)"
    - "traefik.http.routers.portainer-secure.entrypoints=websecure"
    - "traefik.http.routers.portainer-secure.tls=true"

# headers proxy pour que Portainer voie bien HTTPS
-
"traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Proto=https"
-
"traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Host=portainer.rayanec.com"

```

-
"traefik.http.middlewares.portainer-headers.headers.customrequestheaders.X-Forwarded-Port=443"

- "traefik.http.routers.portainer.middlewares=portainer-headers"

=====

MATTERMOST

=====

mm_db:

image: postgres:16

container_name: mm_db

restart: unless-stopped

environment:

POSTGRES_USER: mattermost

POSTGRES_PASSWORD: mattermost

POSTGRES_DB: mattermost

volumes:

- mm_db:/var/lib/postgresql/data

networks:

- infra_net

mattermost:

image: mattermost/mattermost-team-edition:latest

container_name: mattermost

restart: unless-stopped

depends_on:

- mm_db

environment:

MM_SQLSETTINGS_DRIVERNAME: postgres

MM_SQLSETTINGS_DATASOURCE:

postgres://mattermost:mattermost@mm_db:5432/mattermost?sslmode=disable&connect_timeout=10

MM_SERVICESETTINGS_SITEURL: https://chat.rayanec.com

MM_SERVICESETTINGS_TRUSTEDPROXYIPHEADER: X-Forwarded-For

MM_SERVICESETTINGS_TRUSTEDPROXYIP: 172.19.0.0/16

MM_SERVICESETTINGS_ENABLEINSECUREOUTGOINGCONNECTIONS: "false"

MM_SERVICESETTINGS_ENABLEOAUTHSERVICEPROVIDER: "true"

volumes:

- mattermost_data:/mattermost/data

- mattermost_config:/mattermost/config

- mattermost_logs:/mattermost/logs

networks:

- infra_net

labels:

- "traefik.enable=true"



```

- "traefik.http.routers.mattermost.rule=Host(`chat.rayanec.com`)"
- "traefik.http.routers.mattermost.entrypoints=web"
- "traefik.http.services.mattermost.loadbalancer.server.port=8065"
-
"traefik.http.middlewares.mattermost-headers.headers.customrequestheaders.X-Forwarded-Proto=https"
-
"traefik.http.middlewares.mattermost-headers.headers.customrequestheaders.X-Forwarded-Host=chat.rayanec.com"
-
"traefik.http.middlewares.mattermost-headers.headers.customrequestheaders.X-Forwarded-Port=443"
- "traefik.http.routers.mattermost.middlewares=mattermost-headers"

# =====
# TRAEFIK
# =====
traefik:
  image: traefik:2.11
  container_name: traefik
  command:
    - "--providers.docker=true"
    - "--providers.docker.exposedbydefault=false"
    - "--api.dashboard=true"
    - "--entrypoints.web.address=:80"
    - "--entrypoints.websecure.address=:443"
    - "--serversTransport.insecureSkipVerify=true"
  ports:
    - "80:80"
    - "443:443"
    - "8080:8080"
  volumes:
    - "/var/run/docker.sock:/var/run/docker.sock:ro"
  networks:
    - infra_net
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.dashboard.rule=Host(`traefik.rayanec.com`)"
    - "traefik.http.routers.dashboard.entrypoints=websecure"
    - "traefik.http.routers.dashboard.tls=true"
    - "traefik.http.routers.dashboard.service=api@internal"

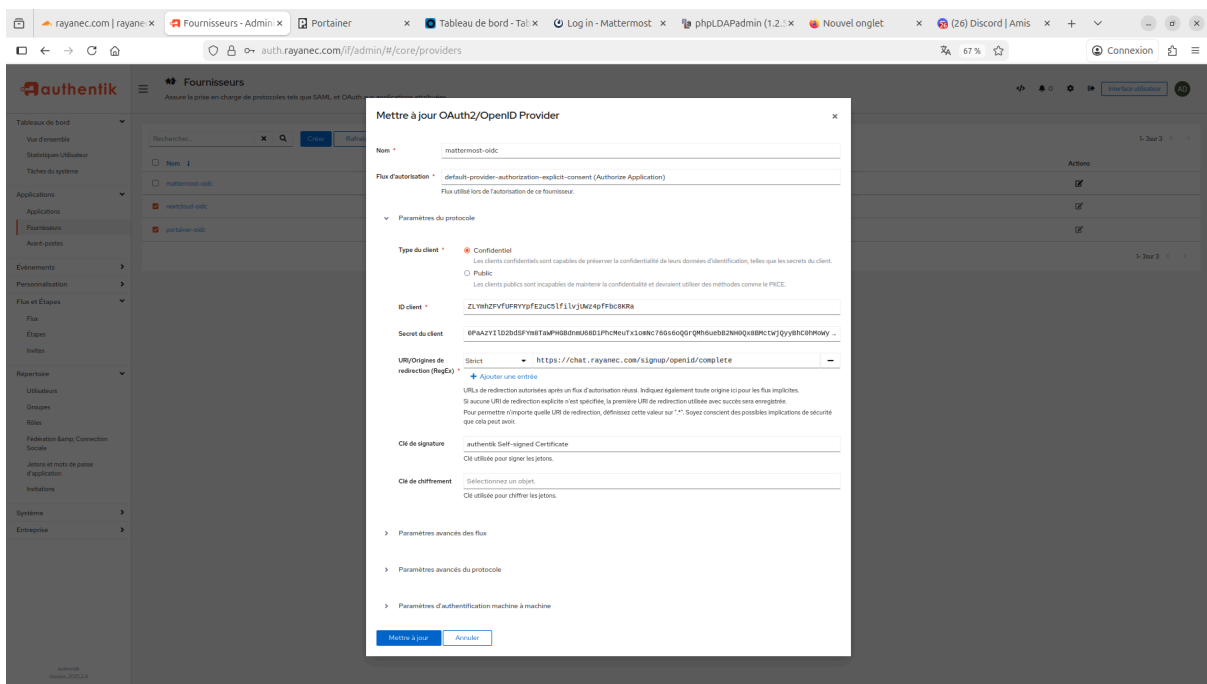
volumes:
  authentik_media:
  authentik_templates:
  authentik_db:
  mm_db:
  mattermost_data:

```

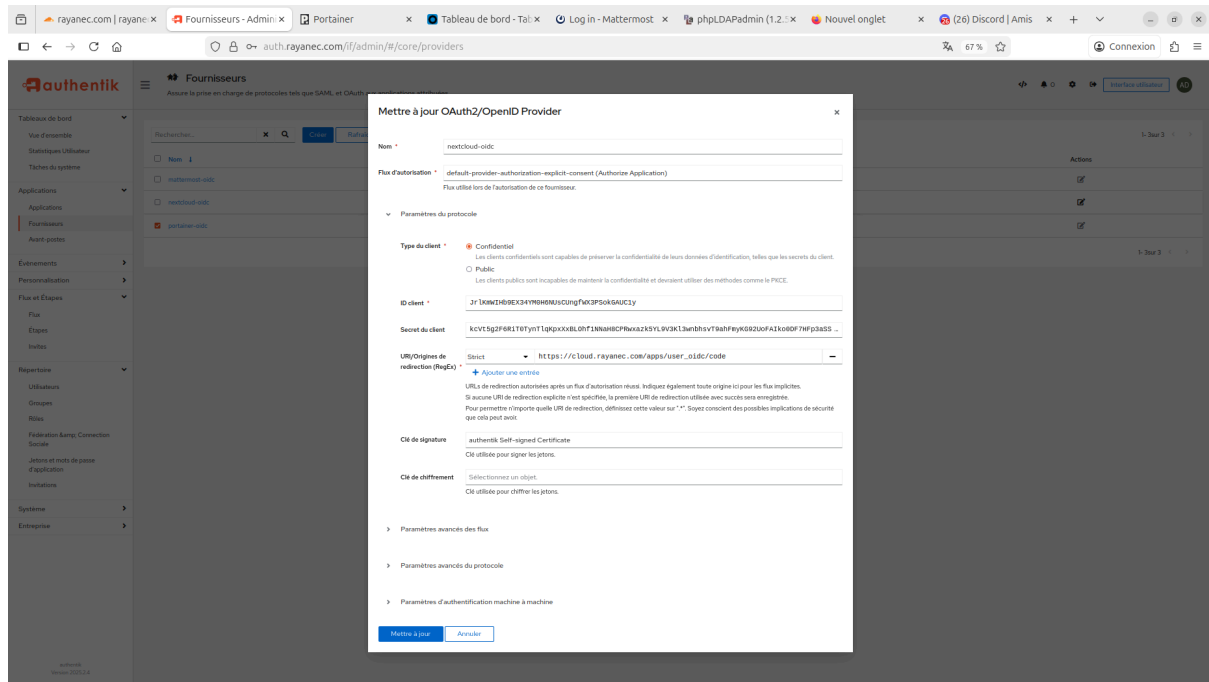
mattermost_config:
mattermost_logs:
portainer_data:

networks:
infra_net:
driver: bridge

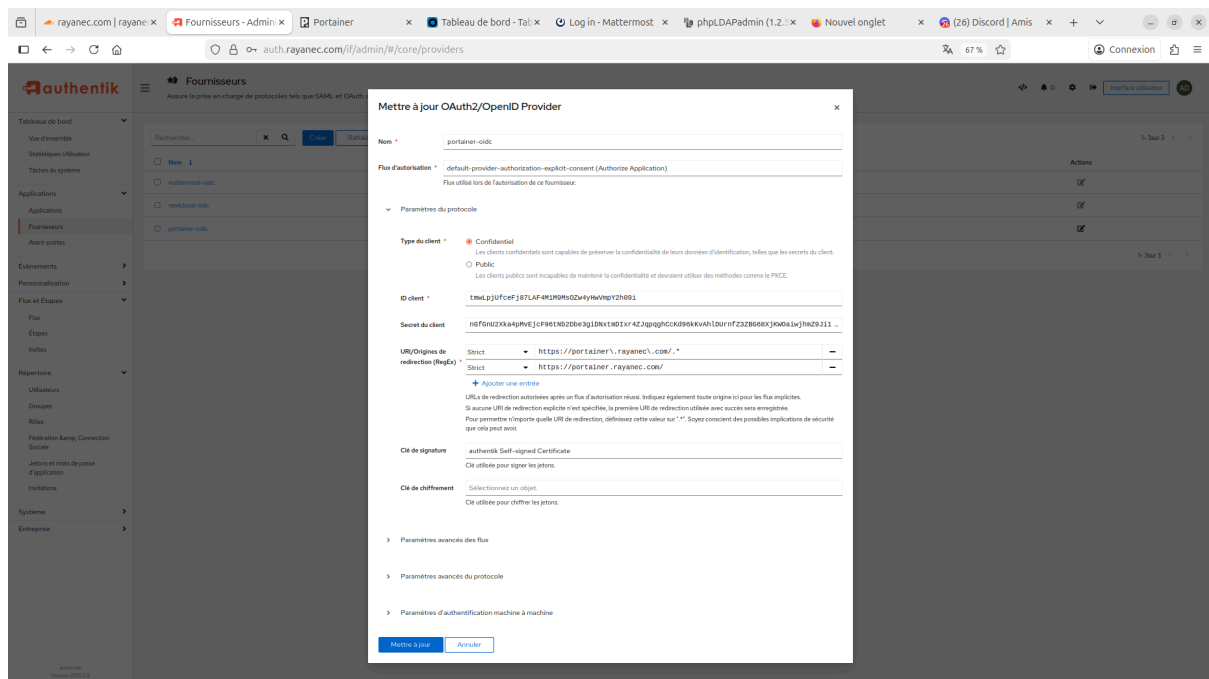
Annexe 2 : Mattermost Provider



Annexe 3 : Nextcloud Provider



Annexe 4 : Portainer Provider



Annexe 5 : Config Mattermost

rayanec.com | rayane.x Applications - Admini Portainer x Tableau de bord - Tai.x System Console - Ma phpLDAPAdmin (1.2.1.x) Nouvel onglet x (26) Discord | Amis x + v

chat.rayanec.com/admin_console/authentication/openid

Preview Mode: Email notifications have not been configured.

< Back to sae-test-team

System Console
@matteradmin

Find settings

Notifications
System-wide Notifications
Emoji
Posts
Content Flagging
File Sharing and Downloads
Public Links
Notices

AUTHENTICATION

Signup
Email
Password
MFA
AD/LDAP
SAML 2.0
OpenID Connect
Guest Access

PLUGINS

Plugin Management
Agents
Calls
Playbooks

OpenID Connect

Select service provider: OpenID Connect (Other)

Follow provider directions for creating an OpenID Application. Most OpenID Connect providers require authorization of all redirect URIs. In the appropriate field, enter "your-mattermost-url/signup/openid/complete" (example: http://domain.com/signup/openid/complete)

Button Name: Login with Authentik
The text that will show on the login button.

Button Color: #145DBF
Specify the color of the OpenID login button for white labeling purposes. Use a hex code with a #-sign before the code.

Discovery Endpoint: https://auth.rayanec.com/application/o/mattermost-well-known/openid-configuration
Enter the URL of the discovery document of the OpenID Connect provider you want to connect with.

Client ID: ZLYmhZfVfURYYpFEZuCsifmJWz4pfbc8KRra
Obtaining the Client ID differs across providers. Please check you provider's documentation.

Client Secret: *****
Obtaining the Client Secret differs across providers. Please check you provider's documentation.

Save

Annexe 6 : Config Nextcloud

rayanec.com | rayane.x Applications - Admini Portainer x OpenID Connect - Pa.x Log in - Mattermost x phpLDAPAdmin (1.2.1.x) Nouvel onglet x (26) Discord | Amis x + v

cloud.rayanec.com/settings/admin/user_oidc

67 %

Connexion

Personnel

Informations personnelles
Sécurité
Notifications
Mobile & bureau
Partage
Apparence et accessibilité
Disponibilités
Flux
Vue privée
Administration
Vue d'ensemble
Support
Paramètres de base
Partage
Sécurité
Personnaliser l'apparence
Intelligence artificielle
Groupware
Privilèges d'administration
Activité
Notifications
Flux
OpenID Connect
Statistiques d'utilisation
Journalisation
Système

OpenID Connect

Allows users to authenticate via OpenID Connect providers.

☐ Active IDMe
☐ Store login tokens

Fournisseurs enregistrés +

authentik

ID Client: jK0mW9H0EX3YMH0H4UClngWx35o4GAUCty
Discovery de la terminaison: https://auth.rayanec.com/application/hnextcloud/well-known/openid-configuration
Backchannel Logout URL: https://cloud.rayanec.com/apps/user_oidc/backchannel-logout/authentik
Redirect URI (its be authorized in the provider client configuration): https://cloud.rayanec.com/apps/user_oidc/code

Configuration du client

Identifiant (max 128 characters): authentik
ID Client: jK0mW9H0EX3YMH0H4UClngWx35o4GAUCty
Secret client: Laisser vide pour conserver l'existant
Warning: If the protocol of the URLs in the discovery content is HTTP, the ID token will be delivered through an insecure connection.

Découverte de la terminaison: https://auth.rayanec.com/application/hnextcloud/well-known/openid-configuration
Custom end session endpoint: (Optional)
Portée: openid email profile
Claims supplémentaires: claim1 claim2 claim3

Mappage des attributs

☐ Enable nested and fallback claim mappings (like "custom:lastname | profile: name | name")

Correspondance de l'ID de l'utilisateur: sub
Correspondance des quotas: quota
Correspondance des groupes: groups

Association d'attributs supplémentaires

Authentication and Access Control Settings

☒ Use unique user ID
By default every user will get a unique user ID that is a hashed value of the provider and user ID. This can be turned off but uniqueness of users across multiple user backends and providers is no longer preserved then.

☐ Use provider identifier as prefix for IDs
To keep IDs in plain text, but also preserve uniqueness of them across multiple providers, a prefix with the providers name is added.

☐ Use group prefixing
This will create and update the users groups depending on the groups claim in the ID token. The format of the groups claim value should be: [spid-"1"; displayName:"group1"; ...3 "group1"; "group2"; ...] or "group1;group2"

Group whitelisted regex:

Only groups matching the whitelisted regex will be created, updated and deleted by the group claim. For example: "[blue]" allows all groups which ID starts with blue

☐ Restrict login for users that are not in any whitelisted group
Users that are not part of any whitelisted group are not created and can not login

☐ Check Secure token on API and WebDAV requests
Do you want to allow API calls and WebDAV requests that are authenticated with an OpenID ID token or access token?

Annuler Mettre à jour le fournisseur



Annexe 7 : Config Portainer

The screenshot displays the Portainer.io web interface in a browser window. The address bar shows the URL `portainer.rayanec.com/#/settings/auth`. The left sidebar contains navigation links: Home, Environment (None selected), Administration, User-related, Environment-related, Registries, Logs, Notifications, Settings, General, Authentication (selected), Edge Compute, and Get Help. The main content area is titled "Automatic user provisioning" and includes a toggle for "Automatic user provisioning" which is currently turned on. Below this, there is a section for "Team membership" and a "Provider" section with four options: Microsoft, Google, Github, and Custom. The "Custom" provider is selected. The "OAuth Configuration" section contains several input fields for Client ID, Client secret, Authorization URL, Access token URL, Resource URL, Redirect URL, Logout URL, User identifier, Scopes, and Auth Style. The "Actions" section at the bottom has a "Save settings" button.

Automatic user provisioning

With automatic user provisioning enabled, Portainer will create user(s) automatically with the standard user role. If disabled, users must be created beforehand in Portainer in order to login.

Automatic user provisioning ☒

The users created by the automatic provisioning feature can be added to a default team on creation.

By assigning newly created users to a team, they will be able to access the environments associated to that team. This setting is optional and if not set, newly created users won't be able to access any environments.

Default team

You have not yet created any teams. Head over to the [Teams view](#) to manage teams.

Team membership

Automatic team membership synchronizes the team membership based on a custom claim in the token from the OAuth provider.

Automatic team membership ☐ [Business Feature](#)

Provider

Microsoft [Business Feature](#) ☐

Google [Business Feature](#) ☐

Github [Business Feature](#) ☐

Custom [Business Feature](#) ☒

OAuth Configuration

Client ID

Client secret

Authorization URL

Access token URL

Resource URL

Redirect URL

Logout URL

User identifier

Scopes

Auth Style

Actions

[Save settings](#)